Written Test – ASP.NET Core & MySQL Software Engineer

Assumptions: MySQL 8.0+, InnoDB, UTC dates, money as DECIMAL(12,2), ASP.NET Core Web API with EF Core.

Section A – MySQL & Database Engineering

Q1. Top 3 Highest-Paid Employees per Department

```
WITH ranked AS (
    SELECT e.Id, e.Name, e.DepartmentId, e.Salary, e.JoiningDate,
           ROW_NUMBER() OVER (
               PARTITION BY e.DepartmentId
               ORDER BY e.Salary DESC, e.JoiningDate ASC, e.Id ASC
           ) AS SalaryRank
    FROM Employee e
)
SELECT Id, Name, DepartmentId, Salary, JoiningDate, SalaryRank
FROM ranked
WHERE SalaryRank <= 3
ORDER BY DepartmentId, SalaryRank;
```

Ties:

- `ROW_NUMBER()` returns exactly 3 rows per department. Equal salaries are broken by JoiningDate, then Id, so the result is deterministic.
- To include ties, use `RANK()` instead: everyone who earns the same as the 3rd person is returned, so a department can have more than 3 rows.
- `DENSE_RANK()` would return the top 3 distinct salary values.

Index: `Employee(DepartmentId, Salary)`

Q2. Seven Consecutive Login Days

```
WITH login_days AS (
    SELECT DISTINCT UserId, LoginDate
    FROM UserLogin
),
numbered AS (
    SELECT UserId, LoginDate,
           ROW_NUMBER() OVER (PARTITION BY UserId ORDER BY LoginDate) AS rn
    FROM login_days
),
islands AS (
    SELECT UserId,
           MIN(LoginDate) AS StartDate,
           MAX(LoginDate) AS EndDate,
           COUNT(*)       AS StreakDays
    FROM numbered
    GROUP BY UserId, DATE_SUB(LoginDate, INTERVAL rn DAY)
)
SELECT UserId, StartDate, EndDate
FROM islands
WHERE StreakDays >= 7
ORDER BY UserId, StartDate;
```

- Duplicate logins on the same date are removed with `DISTINCT` before numbering.
- Overlapping periods are not returned. One row per full streak, so a 10-day streak is returned once with its real start and end dates.

If every 7-day window is needed instead:

```
WITH login_days AS (SELECT DISTINCT UserId, LoginDate FROM UserLogin),
w AS (
    SELECT UserId, LoginDate,
           LAG(LoginDate, 6) OVER (PARTITION BY UserId ORDER BY LoginDate) AS SixBack
    FROM login_days
)
SELECT UserId, SixBack AS StartDate, LoginDate AS EndDate
FROM w
WHERE DATEDIFF(LoginDate, SixBack) = 6;
```

Q3. Customer Order Analytics with Window Functions

```
SELECT o.Id,
       o.CustomerId,
       o.OrderDate,
       o.Amount                        AS CurrentAmount,
       LAG(o.Amount) OVER w            AS PreviousAmount,
       o.Amount - LAG(o.Amount) OVER w AS DifferenceFromPrevious,
       SUM(o.Amount) OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM Orders o
WINDOW w AS (PARTITION BY o.CustomerId ORDER BY o.OrderDate, o.Id)
ORDER BY o.CustomerId, o.OrderDate, o.Id;
```

- Orders placed at the same time are ordered by `Id`, which makes the ordering deterministic.
- For a customer's first order, PreviousAmount and DifferenceFromPrevious are NULL.
- Index: `Orders(CustomerId, OrderDate, Id)`

Q4. Index-Friendly Query Optimization

```
SELECT o.Id, o.OrderDate, o.Amount, o.Status,
       c.Id AS CustomerId, c.Name AS CustomerName,
       p.Id AS ProductId,  p.Name AS ProductName
FROM Orders o
JOIN Customers c ON c.Id = o.CustomerId
JOIN Products  p ON p.Id = o.ProductId
WHERE o.OrderDate >= '2026-06-01'
  AND o.OrderDate <  '2026-07-01'
  AND c.Country = 'BD'
ORDER BY o.OrderDate DESC, o.Id DESC;
```

`YEAR()` and `MONTH()` on the column prevent index use, so I replaced them with a range on the raw column. I also select only the needed columns instead of `SELECT *`.

Indexes:

```
CREATE INDEX IX_Customers_Country_City     ON Customers (Country, City);
CREATE INDEX IX_Orders_Customer_Date_Id    ON Orders (CustomerId, OrderDate, Id);
CREATE INDEX IX_Orders_Date                ON Orders (OrderDate);
```

- Composite column order: equality columns first, then the range column. Once a range is used, the columns after it can't be used to seek.
- `Orders(CustomerId, OrderDate)` is good when there are few BD customers: seek each customer, then scan June.
- `Orders(OrderDate)` is good when June is the selective part. A backward scan also gives `ORDER BY ... DESC` without a filesort.
- The `Products` join uses the primary key.

EXPLAIN ANALYZE:

```
ANALYZE TABLE Orders, Customers;
EXPLAIN ANALYZE <query>;
```

- Before: "Table scan on o", actual rows ≈ the whole table, plus a Sort step.
- After: "Index range scan on o using IX_Orders_Date (reverse)" or "Index lookup using IX_Orders_Customer_Date_Id", far fewer rows examined, no filesort.
- Compare actual time and rows on each node before and after, on production-sized data.

Q5. Secure Dynamic Search and Pagination

```
DELIMITER $$
CREATE PROCEDURE sp_search_orders(
    IN p_customer_name VARCHAR(150), IN p_date_from DATE, IN p_date_to DATE,
    IN p_min_amount DECIMAL(12,2),   IN p_max_amount DECIMAL(12,2),
    IN p_status VARCHAR(20),         IN p_country CHAR(2), IN p_city VARCHAR(100),
    IN p_sort_by VARCHAR(30),        IN p_sort_dir VARCHAR(4),
    IN p_page INT,                   IN p_page_size INT)
BEGIN
    DECLARE v_sort_col VARCHAR(30);
    DECLARE v_sort_dir VARCHAR(4);
    DECLARE v_from  VARCHAR(200)  DEFAULT ' FROM Orders o JOIN Customers c ON c.Id = o.CustomerId';
    DECLARE v_where VARCHAR(2000) DEFAULT ' WHERE 1 = 1';

    -- allowlist for sort column / direction
    SET v_sort_col = CASE LOWER(COALESCE(NULLIF(TRIM(p_sort_by), ''), 'orderdate'))
        WHEN 'orderdate'    THEN 'o.OrderDate'
        WHEN 'amount'       THEN 'o.Amount'
        WHEN 'status'       THEN 'o.Status'
        WHEN 'customername' THEN 'c.Name'
        WHEN 'city'         THEN 'c.City'
        ELSE NULL END;
    IF v_sort_col IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid SortBy';
    END IF;

    SET v_sort_dir = CASE UPPER(COALESCE(NULLIF(TRIM(p_sort_dir), ''), 'DESC'))
        WHEN 'ASC' THEN 'ASC' WHEN 'DESC' THEN 'DESC' ELSE NULL END;
    IF v_sort_dir IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid SortDirection';
    END IF;

    SET @f_page_size = LEAST(GREATEST(COALESCE(p_page_size, 20), 1), 100);
    SET @f_offset    = (GREATEST(COALESCE(p_page, 1), 1) - 1) * @f_page_size;

    -- only add predicates for supplied filters; values go through variables, never SQL text
    IF p_customer_name IS NOT NULL AND TRIM(p_customer_name) <> '' THEN
        SET @f_name = CONCAT(REPLACE(REPLACE(REPLACE(TRIM(p_customer_name),
                      '\\', '\\\\'), '%', '\\%'), '_', '\\_'), '%');
        SET v_where = CONCAT(v_where, ' AND c.Name LIKE @f_name');
    END IF;
    IF p_date_from IS NOT NULL THEN
        SET @f_from = p_date_from;
        SET v_where = CONCAT(v_where, ' AND o.OrderDate >= @f_from');
    END IF;
    IF p_date_to IS NOT NULL THEN
        SET @f_to = DATE_ADD(p_date_to, INTERVAL 1 DAY);
        SET v_where = CONCAT(v_where, ' AND o.OrderDate < @f_to');
    END IF;
    IF p_min_amount IS NOT NULL THEN
        SET @f_min = p_min_amount;
```

```
        SET v_where = CONCAT(v_where, ' AND o.Amount >= @f_min');
    END IF;
    IF p_max_amount IS NOT NULL THEN
        SET @f_max = p_max_amount;
        SET v_where = CONCAT(v_where, ' AND o.Amount <= @f_max');
    END IF;
    IF p_status IS NOT NULL AND p_status <> '' THEN
        SET @f_status = p_status;
        SET v_where = CONCAT(v_where, ' AND o.Status = @f_status');
    END IF;
    IF p_country IS NOT NULL AND p_country <> '' THEN
        SET @f_country = UPPER(p_country);
        SET v_where = CONCAT(v_where, ' AND c.Country = @f_country');
    END IF;
    IF p_city IS NOT NULL AND p_city <> '' THEN
        SET @f_city = p_city;
        SET v_where = CONCAT(v_where, ' AND c.City = @f_city');
    END IF;

    -- total count
    SET @count_sql = CONCAT('SELECT COUNT(*) AS TotalCount', v_from, v_where);
    PREPARE s1 FROM @count_sql; EXECUTE s1; DEALLOCATE PREPARE s1;

    -- page
    SET @page_sql = CONCAT(
        'SELECT o.Id, o.OrderDate, o.Amount, o.Status, c.Id AS CustomerId, ',
        'c.Name AS CustomerName, c.Country, c.City', v_from, v_where,
        ' ORDER BY ', v_sort_col, ' ', v_sort_dir, ', o.Id ', v_sort_dir,
        ' LIMIT ? OFFSET ?');
    PREPARE s2 FROM @page_sql; EXECUTE s2 USING @f_page_size, @f_offset; DEALLOCATE PREPARE s2;

    SET @f_name = NULL, @f_from = NULL, @f_to = NULL, @f_min = NULL, @f_max = NULL,
        @f_status = NULL, @f_country = NULL, @f_city = NULL;
END$$
DELIMITER ;
```

- SQL injection: filter values are bound through variables and never concatenated. Sort column and direction come only from the CASE allowlist. LIMIT/OFFSET are clamped and bound with `?`.
- Index-friendly: a predicate is added only when its filter is supplied (no `(@x IS NULL OR col = @x)`). Dates use a half-open range, and the name uses a prefix LIKE.
- Pagination: separate COUNT query, page size max 100, `Id` as the tie-breaker.
- Performance testing: take the common filter combinations from the logs. Load production-like data, run EXPLAIN ANALYZE for each combination (first page and a deep page), and add or adjust indexes such as `Orders(Status, OrderDate)` and `Customers(Country, City)`. Then load-test the endpoint and watch p95 latency and the slow query log.

Section B – ASP.NET Core & Backend Engineering

Q6. Preventing Duplicate Subscriptions under Concurrency

```
public async Task<IActionResult> Subscribe(SubscribeRequest request, CancellationToken ct)
{
    var now = DateTime.UtcNow;
    await using var tx = await db.Database.BeginTransactionAsync(ct);

    // atomic check + update
    var affected = await db.Users
        .Where(u => u.Mobile == request.Mobile && !u.IsSubscribed)
        .ExecuteUpdateAsync(s => s
```

```
            .SetProperty(u => u.IsSubscribed, true)
            .SetProperty(u => u.SubscribedAt, now), ct);

    if (affected == 0)
    {
        await tx.RollbackAsync(ct);
        var exists = await db.Users.AnyAsync(u => u.Mobile == request.Mobile, ct);
        return exists
            ? Problem(statusCode: 409, title: "Already subscribed")
            : NotFound();
    }

    var userId = await db.Users.Where(u => u.Mobile == request.Mobile)
                               .Select(u => u.Id).SingleAsync(ct);
    db.Subscriptions.Add(new Subscription { UserId = userId, CreatedAt = now });

    try
    {
        await db.SaveChangesAsync(ct);          // UNIQUE(UserId) on Subscriptions
    }
    catch (DbUpdateException ex) when (IsDuplicateKey(ex))   // MySQL error 1062
    {
        await tx.RollbackAsync(ct);
        return Problem(statusCode: 409, title: "Already subscribed");
    }

    await tx.CommitAsync(ct);
    return Ok();
}
```

```
ALTER TABLE Users         ADD UNIQUE INDEX UX_Users_Mobile (Mobile);
ALTER TABLE Subscriptions ADD UNIQUE INDEX UX_Subscriptions_User (UserId);
```

- Race condition: in the original code, all requests read `IsSubscribed = false` before any of them saves, so they all subscribe.
- Fix: the check and the update are one statement (`UPDATE ... WHERE IsSubscribed = 0`). InnoDB locks the row, the other requests wait and then update 0 rows, so only one request gets `affected = 1`.
- The unique constraint on `Subscriptions.UserId` is a second guarantee at the database level.
- Optimistic concurrency (a version column / `DbUpdateConcurrencyException`) also works, but most of the 1,000 requests would fail and need retries. Pessimistic locking (`SELECT ... FOR UPDATE`) works too, but holds locks longer. An in-memory `lock` doesn't work with multiple servers.
- Idempotent: the final state is the same no matter how many times it is called, and only the winning request triggers side effects.
- HTTP responses: `200` subscribed, `409` already subscribed, `404` user not found, `400` invalid input.

Q7. Payment API Design

Data model:

```
Payments
  Id (PK, also sent to provider as idempotency key)
  MerchantId, IdempotencyKey         UNIQUE(MerchantId, IdempotencyKey)
  RequestHash                        SHA-256 of request body
  OrderId
  ActiveOrderId (NULL when Failed)   UNIQUE(MerchantId, ActiveOrderId)
  Amount, Currency
  Status    Pending | Processing | Succeeded | Failed | Unknown
  ProviderReference, FailureReason, Attempts, CreatedAt, UpdatedAt

WebhookEvents
```

```
  Provider, ProviderEventId          UNIQUE(Provider, ProviderEventId)
  EventType, ReceivedAt, ProcessedAt
```

State transitions:

```
Pending -> Processing -> Succeeded
                      -> Failed    (releases ActiveOrderId)
                      -> Unknown   -> Succeeded / Failed (reconciliation or webhook)
```

Endpoint flow:

1. Authenticate the merchant, validate the body, and require the `Idempotency-Key` header (`400` if missing).
2. Hash the request body.
3. Look up by (MerchantId, IdempotencyKey): - hash differs → `422` - still Pending/Processing → `409` + `Retry-After` - finished → return the stored result
4. Insert a Pending row (short transaction). A unique key violation means a duplicate: reload and answer as in step 3, or return `409` if the order already has a live payment.
5. Update Pending → Processing (conditional update).
6. Call the provider outside any DB transaction, with a timeout, sending `Payment.Id` as the provider's idempotency key.
7. Save the result with a conditional update: Succeeded / Failed / Unknown (timeout).
8. Return `201` (succeeded), `402` (declined) or `202` (unknown, poll `GET /api/payments/{id}`).

```
[HttpPost("api/payments")]
public async Task<IActionResult> Create(
    [FromHeader(Name = "Idempotency-Key")] string? key,
    CreatePaymentRequest req, CancellationToken ct)
{
    if (!IsValidKey(key)) return Problem(statusCode: 400, title: "Invalid Idempotency-Key");
    var merchantId = User.FindFirst("merchant_id")!.Value;
    var hash = Sha256(req);

    var existing = await db.Payments.AsNoTracking()
        .FirstOrDefaultAsync(p => p.MerchantId == merchantId && p.IdempotencyKey == key, ct);
    if (existing != null) return Replay(existing, hash);      // 422 / 409 / stored result

    var payment = new Payment
    {
        Id = Guid.CreateVersion7(), MerchantId = merchantId, IdempotencyKey = key!,
        RequestHash = hash, OrderId = req.OrderId, ActiveOrderId = req.OrderId,
        Amount = req.Amount, Currency = req.Currency,
        Status = PaymentStatus.Pending, CreatedAt = DateTime.UtcNow, UpdatedAt = DateTime.UtcNow
    };
    db.Payments.Add(payment);
    try { await db.SaveChangesAsync(ct); }
    catch (DbUpdateException ex) when (IsDuplicateKey(ex))
    {
        db.ChangeTracker.Clear();
        existing = await db.Payments.AsNoTracking()
            .FirstOrDefaultAsync(p => p.MerchantId == merchantId && p.IdempotencyKey == key, ct);
        return existing != null ? Replay(existing, hash)
                                : Problem(statusCode: 409, title: "Duplicate payment for order");
    }

    await db.Payments.Where(p => p.Id == payment.Id && p.Status == PaymentStatus.Pending)
        .ExecuteUpdateAsync(s => s.SetProperty(p => p.Status, PaymentStatus.Processing)
                                  .SetProperty(p => p.Attempts, p => p.Attempts + 1));

    ProviderChargeResult result;
    using var timeout = new CancellationTokenSource(TimeSpan.FromSeconds(15));
```

```
    try
    {
        result = await provider.ChargeAsync(new ProviderChargeRequest(
            payment.Id.ToString("N"), req.OrderId, req.Amount, req.Currency, req.PaymentMethodToken),
            timeout.Token);                          // request token not used after this point
    }
    catch (Exception e) when (e is OperationCanceledException or HttpRequestException)
    {
        result = ProviderChargeResult.Unknown("timeout");
    }

    await ApplyResult(payment.Id, result);           // conditional UPDATE ... WHERE Status IN (...)
    var final = await db.Payments.AsNoTracking().SingleAsync(p => p.Id == payment.Id);

    return final.Status switch
    {
        PaymentStatus.Succeeded => CreatedAtAction(nameof(Get), new { id = final.Id }, final),
        PaymentStatus.Failed    => StatusCode(402, final),
        _                       => AcceptedAtAction(nameof(Get), new { id = final.Id }, final)
    };
}
```

- Idempotency-Key: stored on the payment with a unique index. The same key returns the same response, and the same key with a different body returns `422`.
- Duplicate payment prevention: `UNIQUE(MerchantId, ActiveOrderId)` allows only one pending or successful payment per order. Failed payments set it to NULL, so the order can be retried.
- Transactions: the insert is a short transaction. No transaction or lock is held during the provider call. Every status change is a conditional update, so a payment can never go backwards (e.g. a late "failed" webhook can't undo Succeeded).
- Timeouts / uncertain responses: a timeout is marked Unknown, not Failed, because the charge may have gone through. A background reconciliation job checks Unknown and stuck Processing payments with the provider using the same idempotency key. Retries reuse the key, so the provider never charges twice.
- Distributed lock: useful to reduce duplicate work, e.g. concurrent retries or several reconciler instances. It is not a replacement for database constraints, because a lock can expire (GC pause, network delay), Redis can fail over, and other code paths may not take the lock. The unique indexes and conditional updates are the real guarantee.
- Webhooks: verify the HMAC-SHA256 signature on the raw body with a timestamp (max 5 minutes old) using a constant-time compare, else `401`. Insert into WebhookEvents (unique ProviderEventId) and update the payment in the same transaction. A duplicate event returns `200` and does nothing.
- Status codes: `201`, `202`, `400`, `401/403`, `402`, `404`, `409`, `422`, `500` (ProblemDetails with correlationId).

Q8. Generic Repository / Query Service

```
public abstract class QuerySpec<T> where T : class
{
    private readonly List<Expression<Func<T, bool>>> _criteria = [];
    private readonly List<string> _includes = [];

    public IReadOnlyList<Expression<Func<T, bool>>> Criteria => _criteria;
    public IReadOnlyList<string> Includes => _includes;

    protected abstract IReadOnlyDictionary<string, string> SortMap { get; }  // allowed sort keys
    protected abstract string DefaultSort { get; }

    public string? SortBy { get; init; }
    public bool Descending { get; init; }

    protected void Where(Expression<Func<T, bool>> predicate) => _criteria.Add(predicate);
    protected void Include(string path) => _includes.Add(path);

    public string ResolveOrderBy()
```

```
    {
        var key = string.IsNullOrWhiteSpace(SortBy) ? DefaultSort : SortBy;
        if (!SortMap.TryGetValue(key, out var column))
            throw new InvalidSortException($"Invalid sort field '{key}'.");
        var dir = Descending ? "desc" : "asc";
        return $"{column} {dir}, Id {dir}";
    }
}

public interface IReadRepository<T> where T : class
{
    Task<PagedResult<TResult>> PageAsync<TResult>(QuerySpec<T> spec,
        Expression<Func<T, TResult>> projection, PageRequest page, CancellationToken ct);
    Task<IReadOnlyList<T>> ListEntitiesAsync(QuerySpec<T> spec, int maxItems, CancellationToken ct);
}

public sealed class EfReadRepository<T>(AppDbContext db) : IReadRepository<T> where T : class
{
    public async Task<PagedResult<TResult>> PageAsync<TResult>(QuerySpec<T> spec,
        Expression<Func<T, TResult>> projection, PageRequest page, CancellationToken ct)
    {
        var query = spec.Criteria.Aggregate(db.Set<T>().AsNoTracking(), (q, c) => q.Where(c));

        var total = await query.LongCountAsync(ct);
        if (total == 0) return new([], page.Page, page.PageSize, 0);

        var items = await query.OrderBy(spec.ResolveOrderBy())
                               .Skip(page.Skip).Take(page.PageSize)
                               .Select(projection)
                               .ToListAsync(ct);
        return new(items, page.Page, page.PageSize, total);
    }

    public async Task<IReadOnlyList<T>> ListEntitiesAsync(QuerySpec<T> spec, int maxItems,
CancellationToken ct)
    {
        var query = spec.Criteria.Aggregate(db.Set<T>().AsNoTracking(), (q, c) => q.Where(c));
        query = spec.Includes.Aggregate(query, (q, path) => q.Include(path));
        return await query.OrderBy(spec.ResolveOrderBy())
                          .Take(Math.Clamp(maxItems, 1, 1000)).ToListAsync(ct);
    }
}

public sealed class EmployeeSearchSpec : QuerySpec<Employee>
{
    protected override IReadOnlyDictionary<string, string> SortMap { get; } =
        new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase)
        { ["name"] = "Name", ["salary"] = "Salary", ["joiningDate"] = "JoiningDate" };
    protected override string DefaultSort => "name";

    public EmployeeSearchSpec(EmployeeSearchQuery q)
    {
        SortBy = q.SortBy;
        Descending = q.SortDirection == "desc";
        if (!string.IsNullOrWhiteSpace(q.Name)) Where(e => e.Name.StartsWith(q.Name));
        if (q.DepartmentId is { } d) Where(e => e.DepartmentId == d);
        if (q.MinSalary is { } min) Where(e => e.Salary >= min);
    }
}
```

- Type-safe: filters and projections are `Expression<Func<T, ...>>`, checked at compile time.

- No client expressions: the client only sends values and a sort key. Each spec decides which filters and sort columns exist, and an unknown sort key returns `400`.
- No over-fetching: projection to DTOs, `AsNoTracking`, page size max 100, and count without sorting. Includes are used only when entities are returned.
- Async and cancellation: everything is async and the `CancellationToken` is passed to `ToListAsync` / `CountAsync`, so an aborted request cancels the query.

Q9. Dynamic LINQ Sorting

```
public sealed class InvalidSortException(string message) : ArgumentException(message);

public static class QueryableSortExtensions
{
    public static IOrderedQueryable<T> OrderBy<T>(this IQueryable<T> source, string orderBy,
                                                  IEnumerable<string>? allowedProperties = null)
    {
        if (string.IsNullOrWhiteSpace(orderBy))
            throw new InvalidSortException("Sort expression is required.");

        var clauses = orderBy.Split(',', StringSplitOptions.TrimEntries |
StringSplitOptions.RemoveEmptyEntries);
        if (clauses.Length > 3)
            throw new InvalidSortException("At most 3 sort fields are allowed.");

        var allowed = allowedProperties?.ToHashSet(StringComparer.OrdinalIgnoreCase);
        var expression = source.Expression;

        for (var i = 0; i < clauses.Length; i++)
        {
            var parts = clauses[i].Split(' ', StringSplitOptions.RemoveEmptyEntries);
            var desc = parts.Length switch
            {
                1 => false,
                2 when parts[1].Equals("asc", StringComparison.OrdinalIgnoreCase) => false,
                2 when parts[1].Equals("desc", StringComparison.OrdinalIgnoreCase) => true,
                _ => throw new InvalidSortException($"Invalid sort clause '{clauses[i]}'.")
            };

            if (allowed != null && !allowed.Contains(parts[0]))
                throw new InvalidSortException($"Sorting by '{parts[0]}' is not allowed.");

            var param = Expression.Parameter(typeof(T), "x");
            var property = typeof(T).GetProperty(parts[0],
                BindingFlags.Public | BindingFlags.Instance | BindingFlags.IgnoreCase)
                ?? throw new InvalidSortException($"Unknown sort field '{parts[0]}'.");
            if (!IsScalar(property.PropertyType))
                throw new InvalidSortException($"Field '{parts[0]}' cannot be sorted.");

            var body = Expression.Property(param, property);
            var lambda = Expression.Lambda(body, param);

            var method = (i == 0, desc) switch
            {
                (true, false)  => "OrderBy",
                (true, true)   => "OrderByDescending",
                (false, false) => "ThenBy",
                (false, true)  => "ThenByDescending"
            };

            expression = Expression.Call(typeof(Queryable), method,
```

```
                [typeof(T), body.Type], expression, Expression.Quote(lambda));
        }

        return (IOrderedQueryable<T>)source.Provider.CreateQuery<T>(expression);
    }

    private static bool IsScalar(Type type)
    {
        var t = Nullable.GetUnderlyingType(type) ?? type;
        return t.IsPrimitive || t.IsEnum || t == typeof(string) || t == typeof(decimal)
            || t == typeof(DateTime) || t == typeof(DateTimeOffset) || t == typeof(Guid);
    }
}

// usage
query.OrderBy("Name");
query.OrderBy("Salary desc");
query.OrderBy("JoiningDate");
```

Invalid input (unknown property, wrong direction, navigation property, too many fields) throws `InvalidSortException`. An exception filter turns it into `400 Bad Request` with the error message. It should not fall back silently to a default sort, and it should not become a `500`.

Q10. Request/Response Logging Middleware

```
public sealed class RequestLoggingMiddleware(RequestDelegate next, ILogger<RequestLoggingMiddleware>
logger)
{
    private const int MaxBodyBytes = 16 * 1024;
    private static readonly string[] ExcludedPaths = ["/api/payments", "/api/auth"];

    public async Task InvokeAsync(HttpContext context)
    {
        var correlationId = GetOrCreateCorrelationId(context.Request);   // X-Correlation-ID
        context.TraceIdentifier = correlationId;
        context.Response.OnStarting(() =>
        {
            context.Response.Headers["X-Correlation-ID"] = correlationId;
            return Task.CompletedTask;
        });

        using var scope = logger.BeginScope(new Dictionary<string, object> { ["CorrelationId"] =
correlationId });
        var start = Stopwatch.GetTimestamp();
        var logBodies = logger.IsEnabled(LogLevel.Debug) && !IsExcluded(context.Request.Path);

        string? requestBody = null;
        if (logBodies && context.Request.ContentLength is > 0 and <= 1_048_576)
        {
            context.Request.EnableBuffering();
            var buffer = new byte[MaxBodyBytes];
            var read = await context.Request.Body.ReadAtLeastAsync(buffer, MaxBodyBytes,
                                                                   throwOnEndOfStream: false,
                                                                   context.RequestAborted);
            context.Request.Body.Position = 0;                          // restore for the next
middleware
            requestBody = Redactor.Redact(Encoding.UTF8.GetString(buffer, 0, read));
        }

        var originalBody = context.Response.Body;
        var tee = logBodies ? new CappedTeeStream(originalBody, MaxBodyBytes) : null;
```

```
        if (tee != null) context.Response.Body = tee;

        try
        {
            await next(context);
        }
        catch (OperationCanceledException) when (context.RequestAborted.IsCancellationRequested)
        {
            if (!context.Response.HasStarted) context.Response.StatusCode = 499;   // client closed
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Unhandled exception {Method} {Path}", context.Request.Method,
context.Request.Path);
            if (context.Response.HasStarted) throw;
            context.Response.Clear();
            await Results.Problem(statusCode: 500, title: "An unexpected error occurred.",
                extensions: new Dictionary<string, object?> { ["correlationId"] = correlationId })
                .ExecuteAsync(context);
        }
        finally
        {
            if (tee != null) context.Response.Body = originalBody;
            var elapsed = Stopwatch.GetElapsedTime(start).TotalMilliseconds;

            logger.LogInformation("HTTP {Method} {Path} responded {Status} in {Elapsed:0.0} ms",
                context.Request.Method, context.Request.Path, context.Response.StatusCode, elapsed);
            if (tee != null)
                logger.LogDebug("Request: {Request} | Response: {Response}",
                    requestBody, Redactor.Redact(Encoding.UTF8.GetString(tee.Captured)));
        }
    }
}
```

```
// writes straight through to the real body and keeps a copy of only the first N bytes
public sealed class CappedTeeStream(Stream inner, int capacity) : Stream
{
    private readonly byte[] _buffer = new byte[capacity];
    private int _length;
    public ReadOnlySpan<byte> Captured => _buffer.AsSpan(0, _length);

    public override async ValueTask WriteAsync(ReadOnlyMemory<byte> data, CancellationToken ct = default)
    {
        await inner.WriteAsync(data, ct);
        var take = Math.Min(capacity - _length, data.Length);
        if (take > 0) { data.Span[..take].CopyTo(_buffer.AsSpan(_length)); _length += take; }
    }
    // Write(), Flush(), FlushAsync() forward to inner; CanRead/CanSeek = false
}
```

- Stream buffering/restoration: `EnableBuffering()` makes the request body seekable. I read the first part and set `Position = 0`. The response uses a tee stream instead of a full MemoryStream copy, so streaming isn't broken, and the original stream is restored in `finally`.
- Large payloads: only the first 16 KB is logged. Bodies over 1 MB, file uploads and non-text content types are skipped.
- Cancellation: `RequestAborted` is passed to the read. A client disconnect is logged as 499, not as an error.
- Redaction: JSON keys like password, token, access_token, authorization, cardNumber, cvv and otp are replaced with `***`. Mobile and email are partly masked, and card numbers (Luhn check) are masked. Authorization and Cookie headers are never logged, and payment/auth endpoints are excluded. Body logging is only on at Debug level.

Q11. API Reliability and Observability

1. **Structured logs:** JSON logs with CorrelationId, TraceId, route, status code and duration. Group the 500s by exception and endpoint, then follow one correlation ID through the whole request.
2. **Correlation IDs:** accept or generate `X-Correlation-ID`, return it in the response and forward it to downstream calls.
3. **Metrics:** request rate, error rate and p95/p99 latency per endpoint; CPU, GC, thread pool queue length; DB connection pool usage and wait time; downstream call latency and errors.
4. **Distributed tracing:** OpenTelemetry for ASP.NET Core, HttpClient and EF Core, to see where the time goes (slow query, N+1, waiting for a connection, slow external API).
5. **Database:** slow query log, `performance_schema` statement digests, EXPLAIN ANALYZE on the top queries, and lock waits / deadlocks.
6. **Dependency timeouts:** explicit timeouts on every HttpClient (not the default 100 s), retries only for idempotent calls, and a circuit breaker.
7. **Connection pooling:** check for undisposed contexts, long transactions, pool size × instances vs MySQL max_connections, and sync-over-async. Use `AddDbContextPool` and a short connection timeout.
8. **Load testing:** reproduce with k6/JMeter on production-like data, find the first resource that saturates, fix it, then run the same test again.

First metrics and alerts:

- 5xx error rate > 1% for 5 minutes
- p95 latency above the SLO (e.g. > 500 ms) for 10 minutes
- DB connection pool usage > 80% or connection wait time > 100 ms
- Thread pool queue length increasing
- Dependency timeout/error rate, circuit breaker open
- MySQL slow queries per minute, Threads_running
- Health check failure

Section C – C# & LINQ

Q12. Customer Order Summary

```
var summary = orders
    .GroupBy(o => o.CustomerId)
    .Select(g => new
    {
        CustomerId         = g.Key,
        TotalOrders        = g.Count(),
        HighestOrderAmount = g.Max(o => o.Amount),
        AverageOrderAmount = Math.Round(g.Average(o => o.Amount), 2),
        LatestOrderDate    = g.Max(o => o.OrderDate)
    })
    .ToList();
```

Customers with no orders: when customers and orders are separate collections, start from customers and use `GroupJoin` (left join). Those customers then get TotalOrders = 0 and null for the other values.

```
var summary = customers
    .GroupJoin(orders, c => c.Id, o => o.CustomerId, (c, os) => new { c.Id, Orders = os.ToList() })
    .Select(x => new
    {
        CustomerId         = x.Id,
        TotalOrders        = x.Orders.Count,
        HighestOrderAmount = x.Orders.Count == 0 ? (decimal?)null : x.Orders.Max(o => o.Amount),
        AverageOrderAmount = x.Orders.Count == 0 ? (decimal?)null : Math.Round(x.Orders.Average(o =>
o.Amount), 2),
        LatestOrderDate    = x.Orders.Count == 0 ? (DateTime?)null : x.Orders.Max(o => o.OrderDate)
```

```
    })
    .ToList();
```

Q13. Department Salary Statistics

```
public record DepartmentSalaryStats(int DepartmentId, int Count,
    decimal? Highest, decimal? Lowest, decimal? Average, decimal? Median);

public static List<DepartmentSalaryStats> Calculate(List<Employee> employees, IEnumerable<int>
allDepartmentIds)
{
    var byDept = employees
        .GroupBy(e => e.DepartmentId)
        .ToDictionary(g => g.Key, g => g.Select(e => e.Salary).Order().ToArray());   // sort once

    return allDepartmentIds.Union(byDept.Keys).Order()
        .Select(id =>
        {
            var s = byDept.GetValueOrDefault(id, []);
            if (s.Length == 0) return new DepartmentSalaryStats(id, 0, null, null, null, null);

            var mid = s.Length / 2;
            var median = s.Length % 2 == 1 ? s[mid] : (s[mid - 1] + s[mid]) / 2m;

            return new DepartmentSalaryStats(id, s.Length,
                Highest: s[^1], Lowest: s[0],
                Average: Math.Round(s.Average(), 2),
                Median: median);
        })
        .ToList();
}
```

- Even number of employees: the median is the average of the two middle values.
- Empty department: returned with Count = 0 and null values instead of throwing.
- Complexity: each department is sorted only once, O(n log n) overall. Min, max and median are then read directly from the sorted array.

Section D – API Design & Performance

Q14. Product Search API

`GET /api/products?`
`keyword=phone&minPrice=100&maxPrice=900&categoryIds=1&categoryIds=4&brandIds=2&sortBy=price&sortDirection=`

```
[HttpGet("api/products")]
public async Task<IActionResult> Search([FromQuery] ProductSearchQuery q, CancellationToken ct)
{
    var allowedSorts = new[] { "name", "price", "newest" };
    if (q.PageSize is < 1 or > 100 || q.Page < 1
        || (q.SortBy != null && !allowedSorts.Contains(q.SortBy.ToLower()))
        || (q.SortDirection != null && q.SortDirection is not ("asc" or "desc"))
        || q.MinPrice > q.MaxPrice
        || q.CategoryIds?.Length > 20 || q.BrandIds?.Length > 20
        || q.Keyword?.Length > 100)
        return ValidationProblem();

    var page = q.Page ?? 1;
    var pageSize = q.PageSize ?? 20;
```

```
    var query = db.Products.AsNoTracking().Where(p => p.IsActive);

    if (!string.IsNullOrWhiteSpace(q.Keyword))
        query = query.Where(p => p.Name.Contains(q.Keyword.Trim()));
    if (q.MinPrice is { } min) query = query.Where(p => p.Price >= min);
    if (q.MaxPrice is { } max) query = query.Where(p => p.Price <= max);
    if (q.CategoryIds is { Length: > 0 } cats) query = query.Where(p => cats.Contains(p.CategoryId));
    if (q.BrandIds is { Length: > 0 } brands) query = query.Where(p => brands.Contains(p.BrandId));

    var total = await query.CountAsync(ct);

    var desc = q.SortDirection == "desc";
    query = (q.SortBy?.ToLower() ?? "newest") switch
    {
        "name"  => desc ? query.OrderByDescending(p => p.Name).ThenByDescending(p => p.Id)
                        : query.OrderBy(p => p.Name).ThenBy(p => p.Id),
        "price" => desc ? query.OrderByDescending(p => p.Price).ThenByDescending(p => p.Id)
                        : query.OrderBy(p => p.Price).ThenBy(p => p.Id),
        _       => query.OrderByDescending(p => p.CreatedAt).ThenByDescending(p => p.Id)
    };

    var items = await query
        .Skip((page - 1) * pageSize).Take(pageSize)
        .Select(p => new ProductListItem(p.Id, p.Name, p.Price, p.Category.Name, p.Brand.Name))
        .ToListAsync(ct);

    return Ok(new { items, page, pageSize, totalCount = total });
}
```

- SQL injection: LINQ only, so all values become parameters (`LIKE @p`, `IN (@p0, @p1)`). No raw SQL concatenation.
- Validation: pageSize 1–100, page ≥ 1, sortBy and sortDirection from an allowlist mapped to fixed expressions, minPrice ≤ maxPrice, limits on keyword length and the number of IDs.
- AsNoTracking + projection: no change tracking, and only the needed columns are selected (EF adds the joins).
- Count and page: count runs on the filtered query without ORDER BY or projection. Indexes: `(IsActive, CategoryId, Price)`, `(IsActive, BrandId, Price)`, `(IsActive, CreatedAt, Id)`. FULLTEXT index on Name for keyword search at scale.

Q15. High-Volume API Scenario

- **Don't return all records:** 1 million rows means hundreds of MB per response, high DB load, server memory spikes, slow network and a frozen browser. Users only see 20–50 items. The server always enforces a maximum page size.
- **Offset pagination** (`LIMIT 20 OFFSET 100000`): simple, supports page numbers and total count, but deep pages get slow because MySQL reads and skips all previous rows. Results can also shift when data changes.
- **Keyset pagination** (`WHERE (CreatedAt, Id) < (@lastCreatedAt, @lastId) ORDER BY CreatedAt DESC, Id DESC LIMIT 20`): uses the index directly, so every page is equally fast, and results stay stable. No jumping to page N. Best for infinite scroll.

```
var query = db.Products.AsNoTracking().Where(p => p.IsActive);
if (cursor != null)
    query = query.Where(p => p.CreatedAt < cursor.CreatedAt ||
                             (p.CreatedAt == cursor.CreatedAt && p.Id < cursor.Id));

var rows = await query.OrderByDescending(p => p.CreatedAt).ThenByDescending(p => p.Id)
                      .Take(pageSize + 1)
                      .Select(p => new ProductListItem(p.Id, p.Name, p.Price, p.CreatedAt))
                      .ToListAsync(ct);

var hasMore = rows.Count > pageSize;
```

```
if (hasMore) rows.RemoveAt(pageSize);
var nextCursor = hasMore ? EncodeCursor(rows[^1].CreatedAt, rows[^1].Id) : null;
```

- **Debounced search:** wait about 300 ms after the user stops typing, with a minimum of 2 characters.
- **Cancel obsolete requests:** `AbortController` cancels the previous request on the client. On the server, `CancellationToken` stops the DB query.

```
let controller;
const search = debounce(async term => {
  controller?.abort();
  controller = new AbortController();
  const res = await fetch(`/api/products?keyword=${encodeURIComponent(term)}&pageSize=20`,
                          { signal: controller.signal });
  render(await res.json());
}, 300);
```

- **Caching:** output cache / CDN for popular searches (30–60 s), Redis for counts and category/brand lists, and cache clearing when products change.
- **Indexes:** `(IsActive, CreatedAt, Id)` for keyset, `(IsActive, CategoryId, Price)` and `(IsActive, BrandId, Price)` for filters, FULLTEXT on Name/Description (or a search engine like Elasticsearch).
- **Other:** read replicas for search traffic, rate limiting, response compression, and a cap on deep offsets.